CRYPTORESEARCH · READ FROM THE CODE

The Living System Atlas

A private research desk where models read and argue, code measures and keeps the books, and Eric decides. A visual litepaper and a map of the whole system, each part linked to the source that makes it true.

64
components

72
connections

6
guided paths

164
source links

Outside sources
Services the desk reads from

Intake
How a project enters

Research
Models write graded reports

Scrutiny
Forensics and the prosecution

Decision
Tickets, gates and Eric

Books
Paper results and holdings

Platform Where it runs and how it keeps records

Engineering How the code changes

Read from `main` at commit `0c14b22` of kuaya1/CryptoResearch · November 3, 2026

Contents

I The Litepaper

01 A desk, not a bot

02 The shape of the system

03 How a project enters

04 How research is made

05 Two agents, one argument

06 The exit question

07 The decision

08 The books

09 Who decides what

10 Where it runs

11 How it changes

12 What is not built yet

II The map at a glance

III Six guided paths

IV Component reference

V How to read this atlas

01

A desk, not a bot

CryptoResearch is a private research and paper-trading desk. Models read, argue and write. Code measures, sizes and keeps the books. Eric decides.

Every project on the desk goes through the same steps: it is identified, researched under a frozen prompt, graded by the desk itself, examined for how it could fail, and argued against before anyone decides anything.

The records are plain files written through one checked path. There is no hidden database, and nothing on the desk moves real money.

Models read and argue

Claude, ChatGPT and Gemini research. A second Claude prosecutes. They write words, never amounts.

Code measures and keeps books

Grades, sizing, exits, orders and stops are deterministic JavaScript with exact decimals.

Eric decides

Every decision, every switch that arms automation, and every merge is his.

THE TECHNICAL DETAIL

- The Constitution splits the system in two. A cognitive shell of language models does qualitative research and cross-examination. A deterministic core in JavaScript does sizing, accounting, order states and stops, with exact decimal arithmetic.
- Eric keeps merge authority, capital and the final decision. No loop is ever given those.

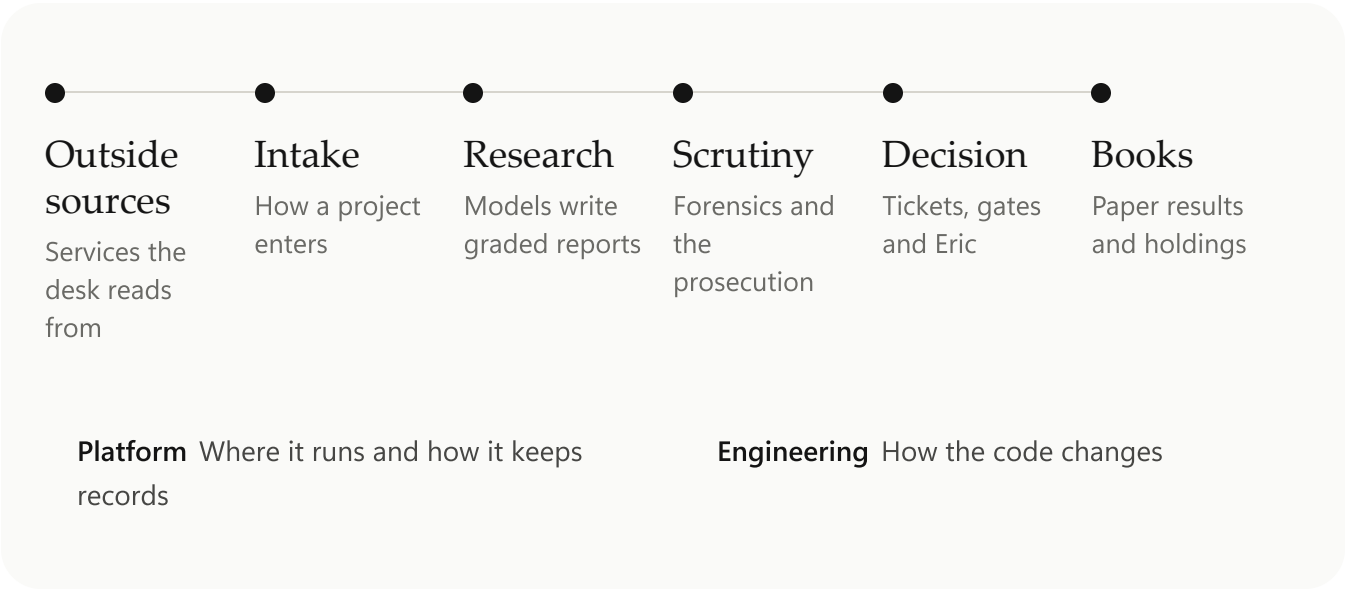
`docs/CONSTITUTION.md:29` `README.md`

02

The shape of the system

Information moves left to right through six stages, on top of a platform that runs it and an engineering loop that changes it.

Outside sources feed intake. Intake feeds research. Research is scrutinized, then decided. Decisions land in books. Each stage writes records the next stage reads, and each record says where it came from.



03

How a project enters

Three doors, all Eric’s: his own link, a scout lead he confirms, or automatic admission inside limits he armed.

The usual door is New research. Eric gives a link, a name or a screenshot; the desk finds CoinGecko candidates and he confirms one.

Every morning a Grok sweep can bring a reading brief and token leads. A brief never starts anything. A lead is checked against CoinGecko and GoPlus, and is admitted only by Eric’s click, or by automatic admission, which is off until he arms it.

On the map: New research · Grok Scout · Scout briefs · Scout leads · Automatic admission · Project records

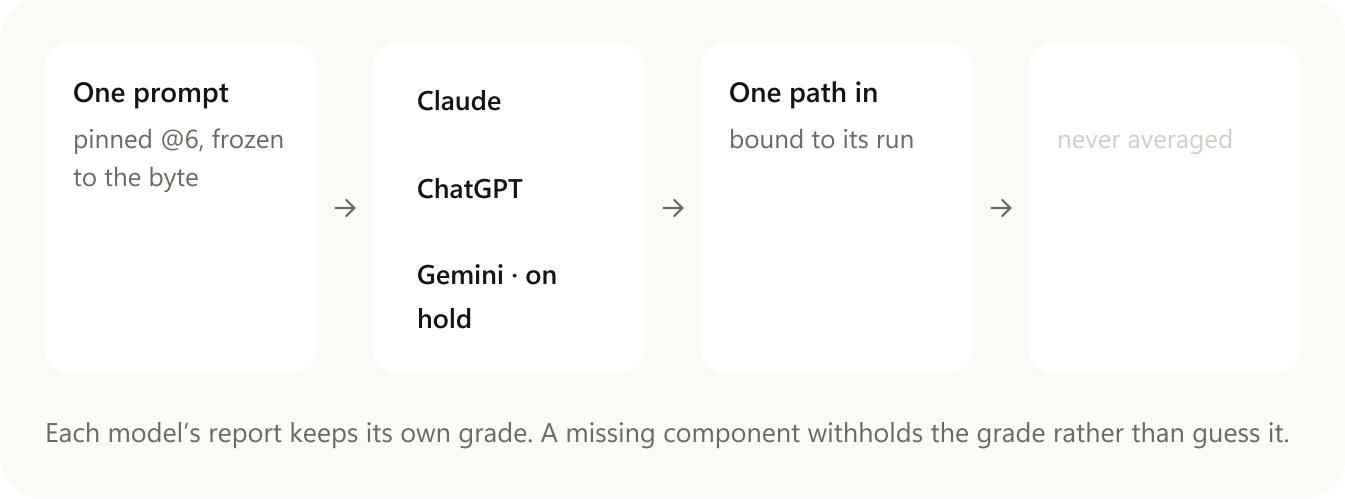
04

How research is made

One prompt, frozen to the byte, is given to every model. The desk, not the model, calculates the grade.

A research run freezes the prompt and an evidence packet of dated prices, fundamentals and desk measurements. Each model gets the identical text.

A report can arrive from a paid API, from an assistant through the connector, or as a pasted answer. All three take the same path in, are bound to their run, and are graded by the same code. An unknown component withholds the grade; it is never filled in.



Each model's report keeps its own grade. A missing component withholds the grade rather than guess it.

THE TECHNICAL DETAIL

- Prompt @6 asks every model three tests: the payment waterfall, holder cash flow per token against supply growth, and one falsifiable thesis.
- Evidence checks read meaning, not field presence, and only flag. They never change a score or a stored report.
- Grades from different models are shown side by side and never averaged.

On the map: Research runs · Pinned prompts · The connector · Report intake · Grading · Evidence checks · Compare & revisions

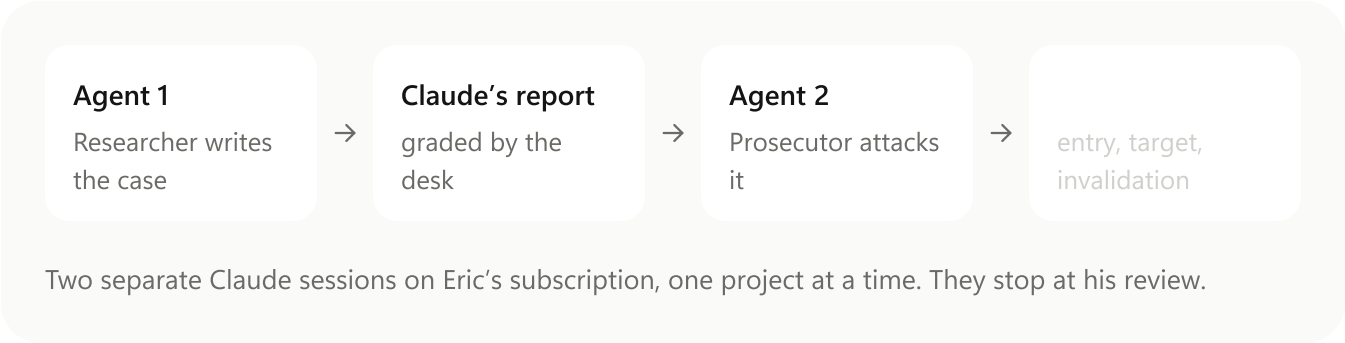
05

Two agents, one argument

Agent 1 makes the case. Agent 2 tries to destroy it. Neither picks a dollar amount.

The pilot runs Claude twice, in separate sessions on Eric’s own subscription. The Researcher writes Claude’s report under prompt @6. The Prosecutor then attacks that report as a short-seller and writes one ticket: an entry zone, an upside target and an invalidation price.

ChatGPT may then challenge the ticket on four tests. Its verdict is shown in words and changes nothing.



Two separate Claude sessions on Eric’s subscription, one project at a time. They stop at his review.

THE TECHNICAL DETAIL

- The runner claims one project at a time and stops at Eric’s review.
- It is built, but no unattended run is recorded yet: the One Sheet says “Operating proof: none yet”.

On the map: Pilot runner · Agent 1 · Researcher · Agent 2 · Prosecutor · ChatGPT challenge

06

The exit question

Before any risk, the desk asks whether a position of that size could actually get out, and at what cost.

Forensic audits investigate how a project could fail: liquidity impairment, operational degradation, delivery inactivity or discontinuation. They are separate from research and change no grade.

The monitor quotes the exit at the position’s size from Jupiter or KyberSwap. A cost is complete only when fees and taxes are known; otherwise it reads “at least”. Every approval needs a reading under 72 hours old.

On the map: Forensic audits · Forensic monitor · Exit readings · Contract forensics

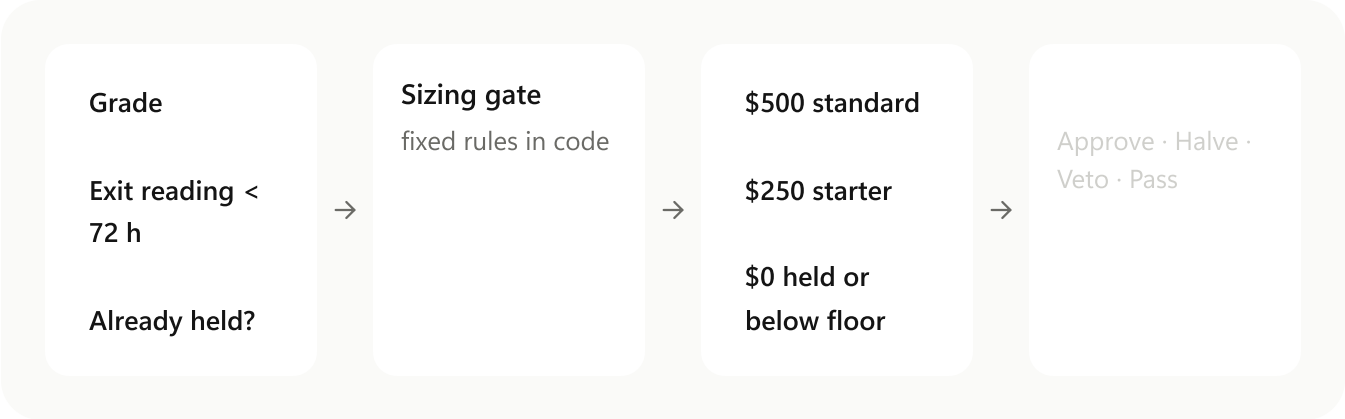
07

The decision

Code suggests the risk. Eric decides. The desk refuses a decision made on stale facts.

The sizing gate turns the grade, the exit reading and whether Eric already holds the asset into a suggested risk: $500, $250 or $0.

On the PM Decision Card Eric approves, halves, vetoes or passes. The desk freezes what he decided on in the same write, and refuses if a newer report arrived, the trigger was breached, the exit reading expired or the card changed.



THE TECHNICAL DETAIL

- The paper mandate can grant paper risk by grade on pilot projects, only inside an authority Eric recorded, expiring within 30 days. It is off by default, and any decision of his own switches it off for that project.

On the map: PM tickets · Sizing gate · Eric's decision · Decision basis · Paper mandate · Off switches

08

The books

Three ways of keeping score, kept apart on purpose, and one ledger of what Eric really holds.

The PM Paper Book records Eric's own grants at the signal price. The Trader's Paper Book simulates real execution: intents, a risk engine, a simulated venue and fills from closed hourly candles. Outcomes measure every eligible report against BTC, traded or not.

Portfolio is separate: Eric's real holdings, recorded by him or from wallets he follows read-only.

PM Paper Book

Eric's own grants at the signal price, compared with BTC. No totals before 20 closed.

Trader's Paper Book

Simulated execution: intents, a risk engine, a file for a venue, fills from closed hours.

Research vs BTC

Every eligible report measured at 30, 60, 90 and 120 days, traded or not.

Portfolio, Eric's real holdings, sits apart from all three.

THE TECHNICAL DETAIL

- The Trader's book stays off until Eric sets its funding, fee and rules.
- An order never times out to "lost". It stays unknown until the venue is asked by its order ID.
- Outcomes reach Eric only. Nothing feeds back into a grade, a rubric or the mandate.

On the map: PM Paper Book · Trader adapter · Risk engine · Trader's Paper Book · Hourly candles · Outcomes · Portfolio · Wallet sync

09

Who decides what

Every automated arrow on the map has an owner who armed it, and one switch that stops it.

Automation runs only inside what Eric switched on. A project stop halts admission, the pilot, mandate grants and openings for that project until he rearms it. The desk-wide pause stops everything.

Automation	Armed by	Default	Stopped by
Grok Scout	Repository variable SCOUT_ENABLED	Off unless set	The variable
Automatic admission	automation.lead_admission, with an expiry	Off	Admission off · project stop
Paid API research	A key, a model, prices, a budget	Not configured	Desk-wide pause · model hold
Scheduled research	research.unattended_enabled	Off	Desk-wide pause
Pilot runner	Eric enables the task and enrolls projects	Task disabled	Project stop
Paper mandate	autonomous_paper_loop and an authority ≤ 30 days	Off	Mandate off · project stop · any decision of his
Trader's Paper Book	trader.book and trader.rules	Unset	Desk-wide pause
Exit quotes	Monitoring per project, or exit_quotes_auto	Off	Monitoring off
Contract forensics	contracts.enabled and compute caps	Off	The setting
Wallet sync	ALCHEMY_API_KEY and a wallet he adds	No wallet	Removing the key or wallet

On the map: Off switches · Paper mandate · Automatic admission · Eric's decision

10

Where it runs

One codebase, two hosts, and files you can open.

The online desk runs on Render from main, with its own records on its own disk. The PC desk runs the same code locally and pulls the only off-Render copy of the records every day.

Every write takes one path: lock, stage, atomic move, verify, log. A save counts only when it reads back verified.

On the map: Online desk · PC desk · Front door · Desk API · Job timers · Journaled writes · Workspace files · Backups

11

How it changes

Supervised, one task at a time. The self-building machine was retired on purpose.

Each change is one task on one branch in Eric’s own Claude Code session, then the test suite, a pull request and Eric’s merge. The trader’s core files are protected by a test.

Merged is not deployed, and deployed is not operating: each is read back on its own.

On the map: Supervised loop · The test check · Protected trader files · Deploy · Agent lane

12

What is not built yet

The honest edge of the system, as the repository itself states it.

- **Ops-1, the Render Cron Job that runs the pilot**

  Defined and suspended, not created.

  `docs/engineering/OPS1_RENDER_CRON.md`

- **An unattended pilot run**

  No operating proof yet; none of milestones B0–B3 recorded.

  `docs/ONE-SHEET.md:74`

- **A real venue**

  None exists. The Trader's venue is a file; live trading would need Eric's signed commit on main.

  `dashboard/core/paper-trader.js:827`

- **GoPlus in research packets**

  Still in monitor mode.

  `dashboard/core/screening.js:53`

- **Real records in the Explorer**

  Refused in code until Eric decides.

  `dashboard/core/explorer.js:537`

- **Gemini on the online desk**

  On hold; nothing new is assigned to it.

  `dashboard/hosted.js:61`

- **Eric's open questions**

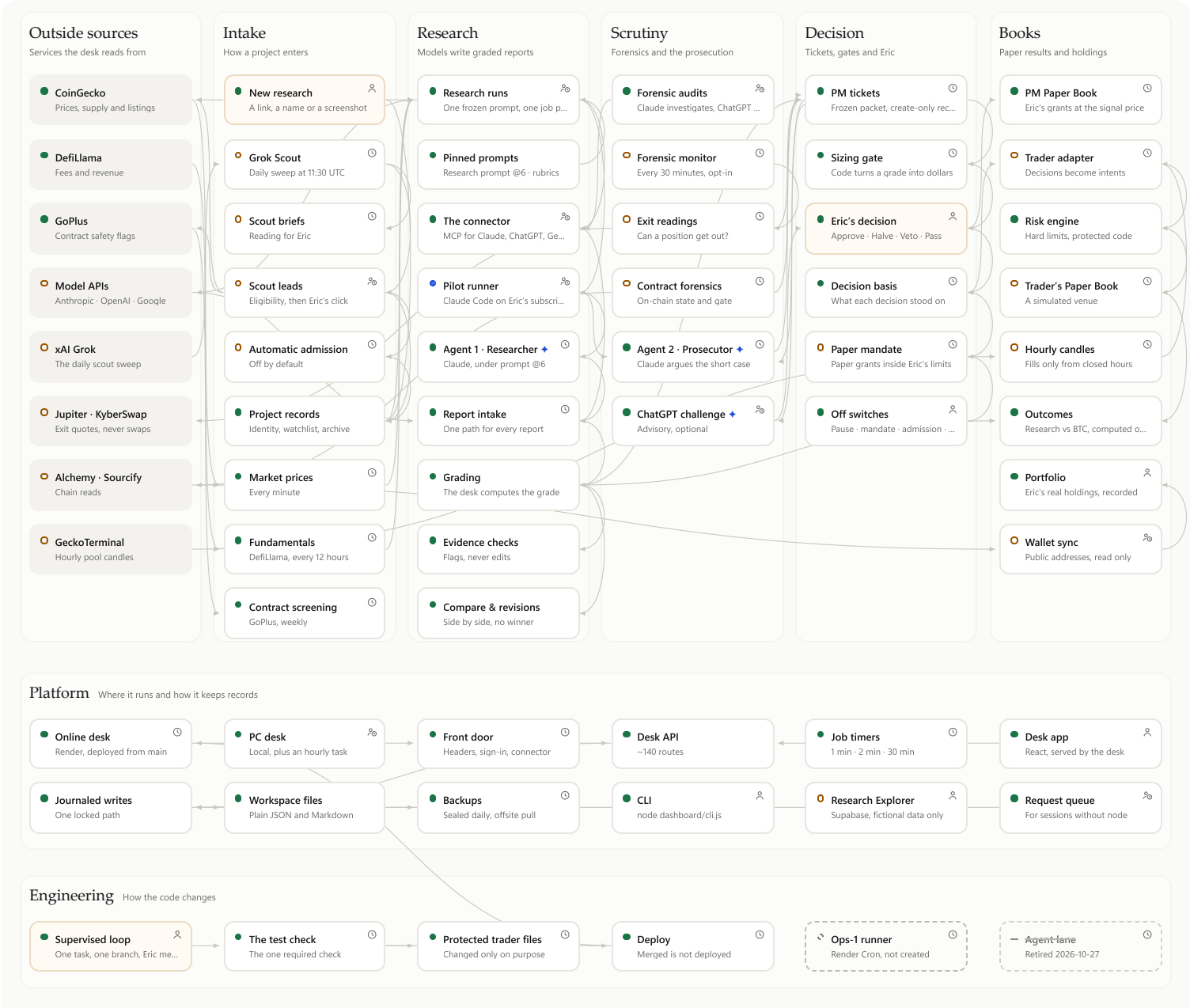
  The Law 1 credential, A5, Invariant 3's wording, the fee-drag baseline.

  `docs/ONE-SHEET.md:76`

On the map: Ops-1 runner · Pilot runner · Research Explorer

II · The map at a glance

Left to right: where information goes. Below: where it runs and how it changes. Connections into the two bands are listed in the component reference.



Operational · Built, off until armed · Partial · Planned · Retired · Eric decides · Automated · Both · AI agent

III · Six guided paths

A link becomes a decision

The main path, from Eric pasting a link to the PM Decision Card.

1. **New research** Eric pastes a link, a name or a screenshot.
2. **CoinGecko** The desk searches CoinGecko for candidates; Eric confirms one.
3. **Project records** A project folder is opened with its verified identity.
4. **Research runs** A research run freezes one prompt and one evidence packet.
5. **Pinned prompts** The prompt is the pinned @6 template, checked byte for byte.
6. **Agent 1 · Researcher** Claude researches and writes the report.
7. **Report intake** The report comes in one way and is bound to its run.
8. **Grading** The desk computes the grade itself.
9. **PM tickets** A ticket opens on Claude's report with a frozen packet.
10. **Agent 2 · Prosecutor** Agent 2 argues the short case and writes the trade plan.
11. **Sizing gate** Code, not a model, suggests the dollar risk.
12. **Eric's decision** Eric approves, halves, vetoes or passes.

The pilot, two agents

How Agent 1 and Agent 2 run on Eric's Claude subscription, one at a time.

1. **Pilot runner** The runner asks the desk's queue what is next and claims one project.
2. **Research runs** The desk prepares the frozen prompt; no paid API is used.
3. **Agent 1 · Researcher** Agent 1 runs in its own Claude session with web tools only.
4. **Report intake** The answer is handed back and saved as Claude's report.
5. **PM tickets** The queue moves the project to the ticket step.
6. **Agent 2 · Prosecutor** Agent 2 runs in a separate session and saves the ticket.
7. **Eric's decision** The pilot stops here. The decision is Eric's.

Scout to research

From the morning Grok sweep to an admitted project.

1. **Grok Scout** At 11:30 UTC, GitHub Actions runs the pinned scout prompt.
2. **xAI Grok** Grok searches the web, one call per lane.
3. **Scout briefs** The brief is stored for Eric to read. It starts nothing.
4. **Scout leads** Token leads are checked against CoinGecko and GoPlus.
5. **Automatic admission** If Eric armed it, eligible leads are admitted inside his caps.
6. **Project records** Otherwise only his click admits a lead.

Can it get out?

The exit reading that every approval needs.

1. **Forensic monitor** For an opted-in project, the monitor sees a quote is due.
2. **Exit readings** It sizes the exit from the stored price, in exact arithmetic.
3. **Jupiter · KyberSwap** Jupiter or KyberSwap answer with a quote. Nothing is swapped.
4. **PM tickets** The reading is frozen into the ticket's packet.
5. **Sizing gate** The gate sizes within the exit-cost limit.
6. **Eric's decision** Approve is refused once the newest reading is over 72 hours old.

A decision on paper

From Eric's approval to fills in the simulated book.

1. **Eric's decision** Eric approves or halves.
2. **Decision basis** What he decided on is frozen in the same write.
3. **PM Paper Book** The PM Paper Book opens a position at the signal price.
4. **Trader adapter** The adapter turns the decision into one trade intent.
5. **Risk engine** The risk engine checks every limit.
6. **Trader's Paper Book** The simulated venue accepts the order.
7. **Hourly candles** Only closed hours after the send can fill it, stop it or hit its target.
8. **Outcomes** Results reach Eric only. Nothing feeds back into a grade.

A code change

How the system itself changes.

1. **Supervised loop** Eric names one task; Claude works on its own branch.
2. **The test check** The test suite runs on the pull request.
3. **Protected trader files** The trader boundary test guards the protected files.
4. **Deploy** Eric merges; Render deploys main.
5. **Online desk** Deployed only when the health check shows the commit.

IV · Component reference

Outside sources Services the desk reads from

CoinGecko Outside service

Operational · On request

The desk's one source of prices and token identity. Every CoinGecko caller shares one rate-limit gate.

- Markets every minute for tracked coins.
- Identity search for new research.
- Contract lookups for scout leads.
- Works without a key; an optional demo key is sent as a header.

Sends to Market prices (prices)

Receives from New research (searches candidates) · Scout leads (checks the listing)

`dashboard/core/market.js:243` `dashboard/core/market.js:305` `dashboard/core/workspace.js:37`

DefiLlama Outside service

Operational · On request

Thirty-day fees, revenue and holders' revenue, read only for projects bound to a DefiLlama slug with a source link.

Sends to Fundamentals (fees, revenue)

`dashboard/core/fundamentals.js:23` `dashboard/core/fundamentals.js:463`

GoPlus Outside service

Operational · On request

Mint, proxy, blacklist and tax flags for each verified contract. Facts only, never a verdict.

Sends to Contract screening (flags)

Receives from Scout leads (checks the contract)

`dashboard/core/screening.js:18` `dashboard/core/screening.js:575`

Model APIs Outside service

Built, off until armed · On request

Claude, ChatGPT and Gemini, called directly only when a model has a key, a model name, prices and a budget. Otherwise research goes through the assistants or a pasted answer.

- Claude: api.anthropic.com Messages, streamed, with web search.
- ChatGPT: api.openai.com Responses, run in the background.
- Gemini: the Interactions API, run in the background; on hold on the online desk.

What has to be true An API key in the environment or secrets file · providers.<model>.model and prices in Settings · A research budget (per run, day, month)

Sends to Report intake (the answer)

Receives from Research runs (paid API call)

`dashboard/core/providers.js:342` `dashboard/core/providers.js:591` `dashboard/core/providers.js:684` `dashboard/core/providers.js:739`

xAI Grok Outside service

Built, off until armed · On request

One web-search call per lane each morning, made from GitHub Actions, never from the desk.

What has to be true Repository variable SCOUT_ENABLED · Secrets XAI_API_KEY, DESK_INGEST_KEY, DESK_INGEST_URL

Sends to Grok Scout (answers the sweep)

`dashboard/scripts/grok-cron.js:31` `.github/workflows/scout-cron.yml:4`

Jupiter · KyberSwap Outside service

Built, off until armed · On request

Keyless quote requests that answer “what would selling this size actually return?”. Nothing is signed, approved or sent.

Receives from Exit readings (quote request)

`dashboard/core/exitquotes.js:8` `dashboard/core/exitquotes.js:12`

Alchemy · Sourcify Outside service

Built, off until armed · On request

Read-only chain access for wallet history and contract state. The contract reader refuses any method outside its read allowlist.

What has to be true ALCHEMY_API_KEY on the host

Sends to Wallet sync (transfers)

Receives from Contract forensics (chain reads)

`dashboard/core/alchemy.js:125` `dashboard/core/contract-rpc.js:287` `dashboard/core/contract-forensics.js:255`

GeckoTerminal Outside service

Built, off until armed · On request

Closed hourly candles from one frozen pool per pair, for the Trader's Paper Book only.

Sends to Hourly candles (closed hours)

`dashboard/core/trader-candles.js:68`

Intake How a project enters

New research Eric

Operational · Eric decides

Eric's front door for a new project. The desk finds CoinGecko candidates, Eric confirms one, and the desk opens the project and a Claude research run.

- One exact link or contract match confirms itself.
- A screenshot is read by Claude only with a key, prices and budget.
- Claude is the only model asked by default.

What has to be true Eric confirms the candidate

Sends to CoinGecko (searches candidates) · Project records (opens the project) · Research runs (requests Claude research)

`dashboard/core/intake.js:159` `dashboard/core/intake.js:335` `dashboard/core/intake.js:609` `dashboard/core/intake.js:582`

Grok Scout Workflow

Built, off until armed · Automated

A GitHub Actions job that sends the pinned scout prompt to Grok over four lanes and posts one brief to the desk. A separate leads call names tokens.

What has to be true SCOUT_ENABLED for the brief · SCOUT_LEADS for token leads

Sends to Scout briefs (posts the brief) · Scout leads (posts token leads)

Receives from xAI Grok (answers the sweep)

`.github/workflows/scout-cron.yml` `dashboard/scripts/grok-cron.js:244` `dashboard/scripts/grok-cron.js:371` `templates/scout/scout-prompt.v1.md`

Scout briefs Records

Built, off until armed · Automated

Briefs stored create-only on the online desk. A brief is something Eric reads; it never becomes research, a ticket or a prompt.

Receives from Grok Scout (posts the brief)

`dashboard/core/scout.js:266` `dashboard/core/scout.js:408`

Scout leads Desk module

Built, off until armed · Both

Token leads checked against CoinGecko's listing and the GoPlus screen under Eric's own thresholds. Only his "Confirm and request research" admits one.

What has to be true leads.enabled and Eric's thresholds · Eric confirms the rows he saw

Sends to CoinGecko (checks the listing) · GoPlus (checks the contract) · Automatic admission (eligible leads) · Project records (Eric's confirm admits)

Receives from Grok Scout (posts token leads)

`dashboard/core/leads.js:148` `dashboard/core/leads.js:593` `dashboard/core/leads.js:948`

Automatic admission Desk module

Built, off until armed · Automated

When Eric arms it, admits up to his daily limit of eligible leads, one pipeline at a time, leaving 15% headroom on every budget downstream.

What has to be true automation.lead_admission.enabled (false by default) · An expiry Eric sets

Sends to Project records (admits inside caps)

Receives from Scout leads (eligible leads)

`dashboard/core/lead-admission.js:308` `dashboard/core/lead-admission.js:378`
`dashboard/core/workspace.js:152`

Project records Records

Operational · On request

One folder per project: its verified identity, research runs, reports, forensics, journal and notes. Archive is reversible; nothing is deleted.

Sends to Research runs (identity)

Receives from New research (opens the project) · Scout leads (Eric's confirm admits) · Automatic admission (admits inside caps)

`dashboard/core/projects.js:300` `dashboard/core/projects.js:456` `WORKSPACE-MAP.md`

Market prices Desk module

Operational · Automated

Keeps the latest price, cap and volume for every tracked coin, with each figure's age.

Sends to Research runs (dated prices in the packet)

Receives from CoinGecko (prices) · Job timers (every minute)

`dashboard/core/market.js:483` `dashboard/server.js:873`

Fundamentals Desk module

● Operational · Automated

Fees and revenue for bound projects, frozen into the research packet. A project without a binding reads "unmapped".

Sends to Research runs (fundamentals in the packet)

Receives from DefiLlama (fees, revenue)

`dashboard/core/fundamentals.js:663` `dashboard/core/fundamentals.js:868`

Contract screening Desk module

● Operational · Automated

GoPlus flags per contract, shown on the desk. Still in monitor mode, so they are not yet put into research packets.

Receives from GoPlus (flags)

`dashboard/core/screening.js:659` `dashboard/core/screening.js:912` `dashboard/core/screening.js:53`

Research Models write graded reports

Research runs Desk module

● Operational · Both

A run freezes one prompt (the same bytes for every model) and one evidence packet, then keeps a job per model with careful paid-attempt bookkeeping and spending limits.

- A job goes to an API, an assistant, or a pasted answer.
- Paid attempts move reserved → dispatched → captured → responded, or uncertain.
- The research tick runs every 2 minutes.

What has to be true Spending limits · research.unattended_enabled for scheduled refreshes (off) · automation.paused stops it all

Sends to Model APIs (paid API call) · The connector (assistant handoff) · Pilot runner (claimed work)

Receives from New research (requests Claude research) · Market prices (dated prices in the packet) · Fundamentals (fundamentals in the packet) · Project records (identity) · Pinned prompts (the frozen prompt) · Job timers (every 2 minutes)

`dashboard/core/research.js:1039` `dashboard/core/research.js:870` `dashboard/core/research.js:926` `dashboard/core/research.js:2372`

Pinned prompts Records

Operational · On request

Prompts, rubrics and schemas are published versions, each pinned by its SHA-256. A byte that differs from the lock is refused, and a published template is never edited.

- @6 adds the payment waterfall, cash flow per token and a falsifiable thesis.
- Thirteen category grades under the @2 scoring profiles.
- Six investment weights, cr-grading@1.

Sends to Research runs (the frozen prompt)

`dashboard/core/prompts.js:532` `dashboard/core/template-lock.js:41` `templates/research-prompt.v6.md` `templates/LOCK.json`

The connector Desk module

Operational · Both

The remote tool server the assistants sign in to with Eric's approval. Each connection is bound to its model and files only that model's work. The desk calls no model here; the assistant does the research.

- 33 tools: research, reports, portfolio, tickets, forensics, challenges.
- Tickets and forensic investigations are Claude's; challenges are ChatGPT's.
- Online desk only.

What has to be true Eric approves each connection with his passphrase

Sends to Report intake (save_report)

Receives from Research runs (assistant handoff) · Forensic audits (Claude and ChatGPT via tools) · Front door (connector)

`dashboard/mcp.js:2199` `dashboard/mcp.js:108` `dashboard/oauth.js:177`

Pilot runner Workflow

Partial · Both

Runs Agent 1 and then Agent 2 as separate Claude sessions, claiming work from the desk and handing back answers. No API key, nothing billed by the desk.

- One claim per project, one agent at a time.
- The Windows task is registered disabled until Eric enables it.
- No unattended run is recorded yet.

What has to be true Eric enrolls each project · Worker and poll token hashes on the desk · Eric enables the task

Sends to Agent 1 · Researcher (starts Agent 1) · Agent 2 · Prosecutor (starts Agent 2)

Receives from Research runs (claimed work)

`dashboard/scripts/pilot.js:765` `dashboard/core/pilot-worker.js:160` `dashboard/core/pilot-claims.js:162` `docs/ONE-SHEET.md:74`

Agent 1 · Researcher AI agent

Operational · Automated

Writes Claude's investment research on one project under the run's frozen prompt. It never writes tickets, never sizes, never decides.

Sends to Report intake (Claude's report)

Receives from Pilot runner (starts Agent 1)

`.claude/agents/forensic-researcher.md` `.claude/commands/pilot-run.md`

Report intake Desk module

Operational · Automated

Every report, from an API, an assistant or a paste, comes in one way: found, parsed, and bound to its run. A field that does not match the run is refused by name.

Sends to Grading (normalized report)

Receives from Model APIs (the answer) · The connector (save_report) · Agent 1 · Researcher (Claude's report)

`dashboard/core/inbox.js:558` `dashboard/core/reports.js:835` `dashboard/core/binding.js:96`

Grading Deterministic engine

Operational · On request

The desk calculates each report's investment grade from its six weighted components. An unknown component withholds the grade instead of guessing it. Grades are never averaged across models.

Sends to Evidence checks (graded report) · Compare & revisions (per-model grades) · PM tickets (Claude's grade is the basis) · Outcomes (every eligible report)

Receives from Report intake (normalized report)

`dashboard/core/scoring.js:248` `dashboard/core/scoring.js:95`

Evidence checks Deterministic engine

Operational · On request

Reads meaning, not field presence: does a code-quality grade name files at a commit, does a liquidity grade cite a dated depth? Flags never change a score.

Receives from Grading (graded report)

`dashboard/core/evidence.js:491` `dashboard/core/textscores.js:117` `dashboard/core/consistency.js:83`

Compare & revisions Deterministic engine

● Operational · On request

Picks each model's current report by one rule, compares numbers only under the same rubric, and labels revisions from the stored records.

Receives from Grading (per-model grades)

`dashboard/core/compare.js:24` `dashboard/core/compare.js:51` `dashboard/core/revisions.js:69`

Scrutiny Forensics and the prosecution

Forensic audits Desk module

● Operational · Both

A separate investigation of whether a project is impaired, degrading, inactive or discontinued. It changes no grade. Receipts are create-only, with fixed checkpoints from 72 hours to 120 days.

Sends to The connector (Claude and ChatGPT via tools)

`dashboard/core/forensics.js:569` `dashboard/core/forensics.js:666` `dashboard/core/forensics.js:841` `templates/forensics/forensic-prompt.v1.md`

Forensic monitor Desk module

○ Built, off until armed · Automated

For projects Eric opts in: guarded product checks, GitHub reads, exit quotes, and the day's checkpoint resolutions. Never a model call.

What has to be true Eric turns monitoring on per project · Eric approves the chains to quote

Sends to Exit readings (due quotes)

Receives from Job timers (every 30 minutes)

`dashboard/core/forensic-monitor.js:365` `dashboard/core/netguard.js:153` `dashboard/core/forensics.js:1016`

Exit readings Deterministic engine

○ Built, off until armed · Automated

Quotes the exit at the position's size every 12 hours. A cost is complete only when the network fee and the transfer tax are known; otherwise it reads "at least".

Sends to Jupiter · KyberSwap (quote request) · PM tickets (exit readings in the packet)

Receives from Forensic monitor (due quotes)

`dashboard/core/exitquotes.js:385` `dashboard/core/forensics.js:1199`

Contract forensics Deterministic engine

○ Built, off until armed · Automated

Reads a contract at a finalized block: code hash, proxy, controllers, supply and authority probes, then a rule-based verdict. It only affects sizing when the gate is set to enforce; today it observes.

What has to be true contracts.enabled and compute caps (off) · tickets.contract_gate.mode = enforce (observe)

Sends to Alchemy · Sourcify (chain reads) · PM tickets (contract verdict)

`dashboard/core/contract-forensics.js:1264` `dashboard/core/contract-gate.js:279` `dashboard/core/workspace.js:173`

Agent 2 · Prosecutor AI agent

● Operational · Automated

Attacks Claude's current report as a short-seller and writes one schema-checked ticket with a trade plan: entry zone, upside target, invalidation. It never chooses a dollar amount.

Sends to PM tickets (the ticket)

Receives from Pilot runner (starts Agent 2) · PM tickets (the frozen packet)

`.claude/agents/junior-trader-prosecutor.md` `templates/tickets/ticket-prompt.v1.md` `templates/schemas/ticket.v1.schema.json`

ChatGPT challenge AI agent

● Operational · Both

ChatGPT may check a saved ticket on four tests. Its verdict shows on the decision card in words and changes no ticket, gate, size or book.

Sends to Eric's decision (advice in words)

Receives from PM tickets (a saved ticket)

`dashboard/core/ticket-challenges.js:240` `dashboard/core/ticket-challenges.js:356` `templates/tickets/challenge-prompt.v1.md`

Decision Tickets, gates and Eric

PM tickets Desk module

● Operational · Automated

Opens a ticket from Claude's current report, freezes the evidence packet and the Prosecutor's prompt, and saves the Prosecutor's ticket as a receipt that never changes.

Sends to Agent 2 · Prosecutor (the frozen packet) · ChatGPT challenge (a saved ticket) · Sizing gate (grade, exits, held)

Receives from Grading (Claude's grade is the basis) · Exit readings (exit readings in the packet) · Contract forensics (contract verdict) · Agent 2 · Prosecutor (the ticket)

`dashboard/core/tickets.js:644` `dashboard/core/tickets.js:1360` `dashboard/core/tickets.js:2756`

Sizing gate Deterministic engine

Operational · Automated

Fixed rules turn the grade, the exit reading and whether Eric holds the asset into an allowed ticket type and a suggested risk: $500 standard, $250 starter, $0 below the floor.

- Below 48 on an asset Eric does not hold, no Prosecutor runs (pregate).
- A ticket whose invalidation cannot be checked against a fresh price is held to $0.
- A trigger already breached at save writes nothing.

Sends to Eric's decision (suggested risk)

Receives from PM tickets (grade, exits, held)

`dashboard/core/tickets.js:416` `dashboard/core/tickets.js:510` `dashboard/core/tickets.js:1278`

Eric's decision Eric

Operational · Eric decides

The PM Decision Card. Eric approves the suggested risk, halves it, vetoes or passes. The desk refuses a stale decision: an expired exit reading, a newer report, an invalidated trigger or a changed card.

- Approve and Halve need an exit reading under 72 hours old.
- The card names what changed when it refuses.

What has to be true Eric alone

Sends to Decision basis (frozen in the same write) · PM Paper Book (Eric's grants)

Receives from ChatGPT challenge (advice in words) · Sizing gate (suggested risk)

`dashboard/core/tickets.js:1843` `dashboard/core/tickets.js:1811`
`dashboard/web/src/views/inspector/PMDecisionCard.jsx`

Decision basis Records

Operational · Automated

Freezes the facts behind every decision in the same write as the decision, with a SHA-256 ledger checked on every read.

Sends to Trader adapter (approve or halve)

Receives from Eric's decision (frozen in the same write) · Paper mandate (grants inside limits)

`dashboard/core/decision-basis.js:147` `dashboard/core/decision-basis.js:285`

Paper mandate Desk module

Built, off until armed · Automated

When Eric arms it, grants paper risk on pilot projects by grade (50–54 a halve, 55 and up an approve), within an expiry of at most 30 days. Any decision of his own switches it off for that project.

What has to be true automation.autonomous_paper_loop (false by default) · An authority generation Eric records · An expiry within 30 days

Sends to Decision basis (grants inside limits)

Receives from Off switches (stops)

`dashboard/core/tickets.js:2639` `dashboard/core/mandate-authority.js:165` `dashboard/core/workspace.js:134`

Off switches Interface

Operational · Eric decides

One page of off-switches. A project stop halts every automated arrow on that project until Eric rearms it; the desk-wide pause is separate.

Sends to Paper mandate (stops)

`dashboard/core/desk-controls.js:77` `dashboard/core/mandate-authority.js:176`

Books Paper results and holdings

PM Paper Book Records

Operational · Automated

One paper position per approve or halve of Eric's, voided on Undo, compared with BTC. No totals before 20 closed positions. Mandate grants never enter it.

Receives from Eric's decision (Eric's grants)

`dashboard/core/paper.js:123` `dashboard/core/paper.js:175` `dashboard/core/paper.js:375`

Trader adapter Deterministic engine

Built, off until armed · Automated

The only caller of the Trader's book. Turns an approve or halve into one trade intent (entry, stop, target, risk), and closes on Undo, the horizon's end or an invalidation.

What has to be true trader.book funding and fee · trader.rules: stop distance, notional, allowlist

Sends to Risk engine (intent)

Receives from Decision basis (approve or halve) · Hourly candles (fills, stops, targets) · Job timers (every 30 minutes)

`dashboard/core/trader-desk.js:781` `dashboard/core/trader-desk.js:818` `dashboard/core/trader-desk.js:264`

Risk engine Deterministic engine

● Operational · On request

Checks every intent before it is sent: 2% of the book per trade, 4% drawdown in 24 hours, 20 bps slippage, the allowlist, the stop distance, the exit cap, and a halt while any order is unresolved.

Sends to Trader's Paper Book (checked intent)

Receives from Trader adapter (intent)

`dashboard/core/trade-risk.js:135` `dashboard/tests/trader-boundary.test.js:30`

Trader's Paper Book Records

○ Built, off until armed · Automated

A simulated book with a hash-chained journal. Its "venue" is a file. An order never times out to lost: it stays unknown until the venue is asked.

Sends to Outcomes (book results)

Receives from Risk engine (checked intent)

`dashboard/core/paper-trader.js:867` `dashboard/core/paper-trader.js:827`
`dashboard/core/paper-trader.js:927` `dashboard/core/paper-trader.js:1077`

Hourly candles Desk module

○ Built, off until armed · Automated

Closed hourly candles for each pair. A missing hour is never filled in. Asks nothing until the book is funded.

Sends to Trader adapter (fills, stops, targets)

Receives from GeckoTerminal (closed hours)

`dashboard/core/trader-candles.js:522` `dashboard/core/trader-candles.js:186`

Outcomes Deterministic engine

● Operational · On request

Each eligible report's return against BTC at 30, 60, 90 and 120 days, beside its own verdict. Never a ranking, never "most accurate", nothing stored.

Receives from Grading (every eligible report) · Trader's Paper Book (book results)

`dashboard/core/report-outcomes.js:220` `dashboard/core/grade-outcome-view.js:205`

Portfolio Records

● Operational · Eric decides

Eric's own ledger of real holdings and cost, kept apart from every paper book. Nothing here places a trade.

Receives from Wallet sync (verified events)

`dashboard/core/portfolio.js:222` `dashboard/core/portfolio.js:490`

Wallet sync Desk module

Built, off until armed · Both

Follows public addresses Eric adds, twice a day. Only final, verified events become ledger entries; a possible swap waits for his decision.

What has to be true ALCHEMY_API_KEY · A wallet Eric adds himself

Sends to Portfolio (verified events)

Receives from Alchemy · Sourcify (transfers)

`dashboard/core/walletsync.js:506` `dashboard/core/wallets.js:1121` `dashboard/core/wallets.js:266`

Platform Where it runs and how it keeps records

Online desk Desk module

Operational · Automated

The same code, deployed from main to Render, with its own records on a Render disk. It refuses to start without a disk, a passphrase and a host name.

Sends to Front door (every request)

Receives from Deploy (deploys main)

`server.js` `dashboard/hosted.js:466` `dashboard/hosted.js:174`

PC desk Desk module

Operational · Both

The desk on Eric's PC, bound to this machine only. An hourly Windows task runs due jobs and pulls the only off-Render copy of the records.

`dashboard/core/schedule.js:30` `dashboard/scripts/scheduled-run.cmd`

Front door Desk module

Operational · Automated

Everything a request meets before the desk: security headers, the health check, host checks, the pilot and connector routes, scout ingest, then the passphrase sign-in.

Sends to Desk API (signed in) · The connector (connector)

Receives from Online desk (every request)

`dashboard/frontdoor.js:125` `dashboard/hosted.js:337`

Desk API Desk module

Operational · On request

The signed-in API behind the app. Every call carries a per-launch token.

Sends to Journaled writes (changes)

Receives from Front door (signed in) · Desk app (reads and actions)

`dashboard/server.js:974` `dashboard/server.js:523`

Job timers Desk module

Operational · Automated

Market every minute, research every 2 minutes, and every 30 minutes: snapshot, reviews, backup, fundamentals, screening, forensics, trader, leads.

Sends to Market prices (every minute) · Research runs (every 2 minutes) · Forensic monitor (every 30 minutes) · Trader adapter (every 30 minutes)

`dashboard/core/jobs.js:41` `dashboard/core/jobs.js:294`

Desk app Interface

Operational · Eric decides

Desk, Review, Portfolio, Research and Outcomes, plus the inspector and each project's page. Opening a page never writes.

Sends to Desk API (reads and actions)

`dashboard/web/src/lib/router.js` `dashboard/web/src/views/watchlist/StatusStrip.jsx`

Journaled writes Deterministic engine

Operational · On request

Every change goes through one operation: lock, stage, atomic move, verify, log. A save counts only when it reads back verified.

Sends to Workspace files (atomic writes)

Receives from Desk API (changes) · CLI (changes) · Request queue (changes)

`dashboard/core/ops.js:665` `dashboard/core/ops.js:409` `dashboard/core/fsx.js:49`

Workspace files Records

Operational · On request

No hidden database. Projects, portfolio, market data, inbox, logs and backups are folders of files.

Sends to Backups (daily)

Receives from Journaled writes (atomic writes)

`WORKSPACE-MAP.md` `dashboard/core/workspace.js:355`

Backups Desk module

Operational · Automated

A sealed daily backup on the desk's disk, and an offsite package the PC pulls with its own key. A restored copy asks no provider anything until released.

Receives from Workspace files (daily)

`dashboard/core/exporter.js:815` `dashboard/core/offsite.js:197` `dashboard/core/offsite.js:310`

CLI Interface

Operational · Eric decides

Every record change from a terminal: doctor, research, tickets, forensics, paper, wallets, backups, tests.

Sends to Journaled writes (changes)

`dashboard/cli.js`

Research Explorer Desk module

Built, off until armed · Eric decides

An optional one-way mirror into Supabase. Only fictional fixtures are sent; sending real records is refused in code until Eric decides.

`dashboard/core/explorer.js:537` `dashboard/core/explorer.js:490`

Request queue Records

Operational · Both

A session that cannot run the CLI leaves a request file; the desk processes it and writes a receipt.

Sends to Journaled writes (changes)

`dashboard/core/requests.js:280`

Engineering How the code changes

Supervised loop Eric

Operational · Eric decides

One task at a time on its own branch in Eric's Claude Code session, the test suite, a pull request, and Eric's own merge.

Sends to The test check (pull request)

`docs/CONSTITUTION.md:18`

The test check Workflow

● Operational · Automated

GitHub runs the test suite on every pull request; it is the only check main requires.

Sends to Protected trader files (boundary test) · Deploy (Eric merges)

Receives from Supervised loop (pull request)

`.github/workflows/test.yml` `dashboard/cli.js:225`

Protected trader files Deterministic engine

● Operational · Automated

The risk engine, both paper books, the decimal maths and their tests are protected: a test fails if they change outside a deliberate pull request.

Receives from The test check (boundary test)

`dashboard/tests/trader-boundary.test.js:30`

Deploy Workflow

● Operational · Automated

Render deploys main. A merge counts as deployed only once the health check shows its commit, read back at a stated time.

Sends to Online desk (deploys main)

Receives from The test check (Eric merges)

`docs/ONE-SHEET.md:73`

Ops-1 runner Workflow

◌ Planned · Automated

The planned hourly Render Cron Job that will run the pilot after the PC's supervised proof. Defined and suspended; not yet created.

`docs/engineering/OPS1_RENDER_CRON.md` `dashboard/scripts/ops1-preflight.js`

Agent lane Workflow

– Retired · Automated

The self-building agent lane, the PM builder and the engineering connector were deleted on purpose. Nothing in the repository builds, reviews or merges on a schedule.

`docs/CONSTITUTION.md:15` `.github/rulesets/README.md`

V · How to read this atlas

Every component, connection and claim was read from the repository's code and documents. Each source link opens the exact line on GitHub, pinned to commit `0c14b22`, so it keeps pointing at what was read even as the code moves on.

A build step checks every cited function, constant or line against the files and refuses to produce the atlas if any is missing, or if a connection names a part that does not exist.

Status	Means
● **Operational**	Built and running in the code on main
○ **Built, off until armed**	Built; waits for a key, a setting or Eric's switch
◐ **Partial**	Built in part, or built with no operating proof yet
◌ **Planned**	Defined in a document, not built or not created
— **Retired**	Removed on purpose; must not be rebuilt
Automated	Runs on a timer or by itself once armed
Eric decides	Happens only by Eric's own click, command or merge
Both	Eric starts or arms it; the desk carries it out
On request	Runs only when another part calls it

"Operational" means built in the code on main. It does not mean deployed or proven in operation: the repository's One Sheet keeps merged, deployed and operating apart, and states "Operating proof: none yet".